

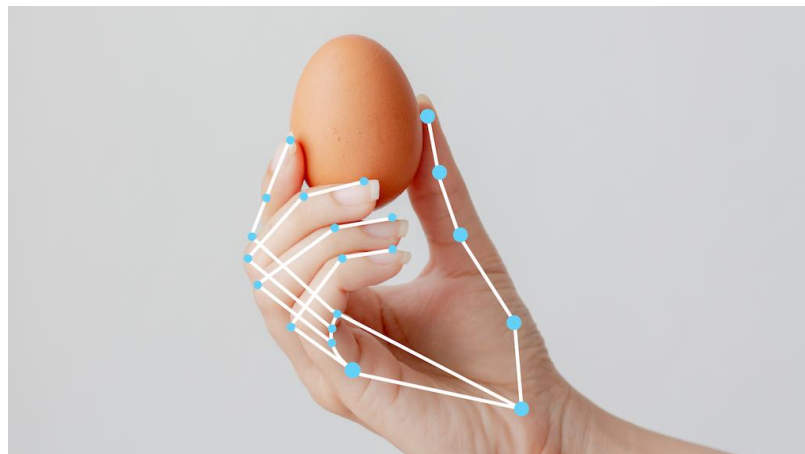


Objetivos de la práctica

- Conocer el paquete de productos MediaPipe de Google para el desarrollo de aplicaciones que hagan uso de técnicas de inteligencia artificial y aprendizaje automático.
- Utilizar MediaPipe para detección de manos y gestos en imágenes o videos.
- Desarrollar en Python una interfaz mediante gestos de las manos para el control de un minijuego.

Introducción

El **reconocimiento de gestos** permite generar interfaces que interpretan gestos humanos como comandos. En otras palabras, es la habilidad de un ordenador para interpretar gestos generados por un operador humano y ejecutar comandos basándose en ellos. En esta práctica se van a utilizar MediaPipe Hand Landmarker para detectar los puntos de referencia de la mano y poder interpretar los gestos que esta realiza como entrada de comandos.



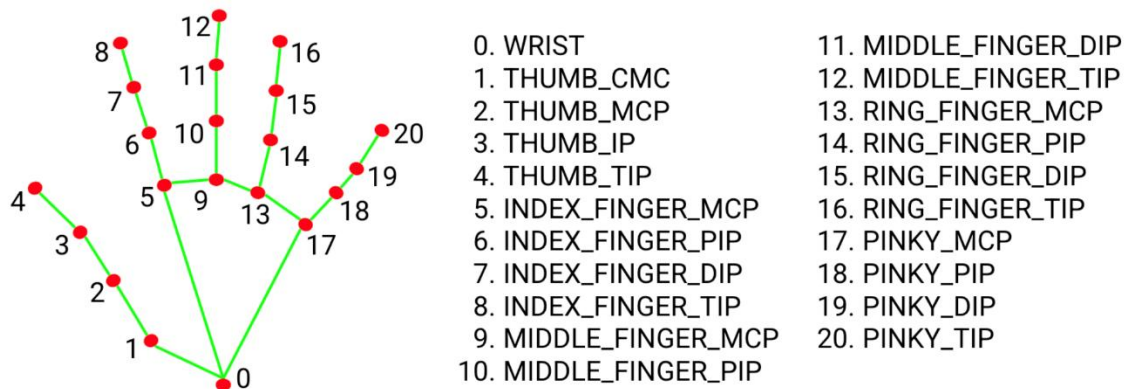
MediaPipe

MediaPipe es un proyecto de código abierto de Google que proporciona un conjunto de bibliotecas y herramientas para poder aplicar rápidamente técnicas de inteligencia artificial y aprendizaje automático a tus aplicaciones. Como se verá mas adelante cuenta con un gran número de tareas ya implementadas. Se invita al alumno a que visite la web <https://ai.google.dev/edge/mediapipe/solutions/> y revise las tareas que se encuentran ya implementadas. En esta práctica nos centraremos en la detección de puntos de referencia de la mano, pero existen tareas de detección de puntos de referencia faciales, clasificación de imágenes, detección de posición de tronco y extremidades, etc.

Hand Landmarker usa un paquete de modelos con dos modelos empaquetados:

- modelo de detección de palma.
- modelo de detección de puntos de referencia de la mano.

El paquete de modelos de puntos de referencia de la mano detecta la localización de los puntos clave de 21 coordenadas de mano en las regiones de la mano detectadas. El modelo se entrenó con alrededor de 30,000 imágenes del mundo real, así como con varios modelos de mano sintética renderizados sobre varios fondos.



El paquete de modelos del punto de referencia de la mano contiene un modelo de detección de la palma y uno de detección de puntos de referencia de la mano. El modelo de detección de la palma localiza las manos dentro de la imagen de entrada, y el modelo de detección de puntos de referencia de la mano identifica puntos de referencia específicos de la mano en la imagen recortada de la mano que define el modelo de detección de la palma.

Como la ejecución del modelo de detección de palma requiere mucho tiempo, cuando está en modo de ejecución de video o transmisión en vivo, el marcador manual usa el cuadro de límite definido por el modelo de puntos de referencia de la mano en un fotograma a fin de localizar la región de las manos para los fotogramas posteriores. El marcador de mano solo vuelve a activar el modelo de detección de la palma si el modelo de puntos de referencia de la mano ya no identifica la presencia de manos o no puede rastrearlas dentro del marco. Esto reduce la cantidad de veces que se activa el modelo de detección de palma.



Detección de manos en una imagen



La primera tarea que vamos a realizar para familiarizarnos con la plataforma es la detección de las manos y puntos característicos de estas en una imagen. En la práctica emplearemos el lenguaje de programación Python 3.12 y han de instalarse preferiblemente en un entorno virtual los paquetes:

```
pip install opencv-python
pip install mediapipe==0.10.21
pip install pygame
```

dos primeros serán necesarios para el reconocimiento de manos y el último nos será útil para el desarrollo de un minijuego controlado por gestos mediante la webcam de nuestro portátil. Para verificar que todo funciona haremos uso de un ejemplo muy simple que detecta los puntos característicos de la mano en una imagen:

```
import cv2
import mediapipe as mp

mp_drawing = mp.solutions.drawing_utils
mp_hands = mp.solutions.hands

with mp_hands.Hands(
    static_image_mode=True,
    max_num_hands=2,
    min_detection_confidence=0.5) as hands:

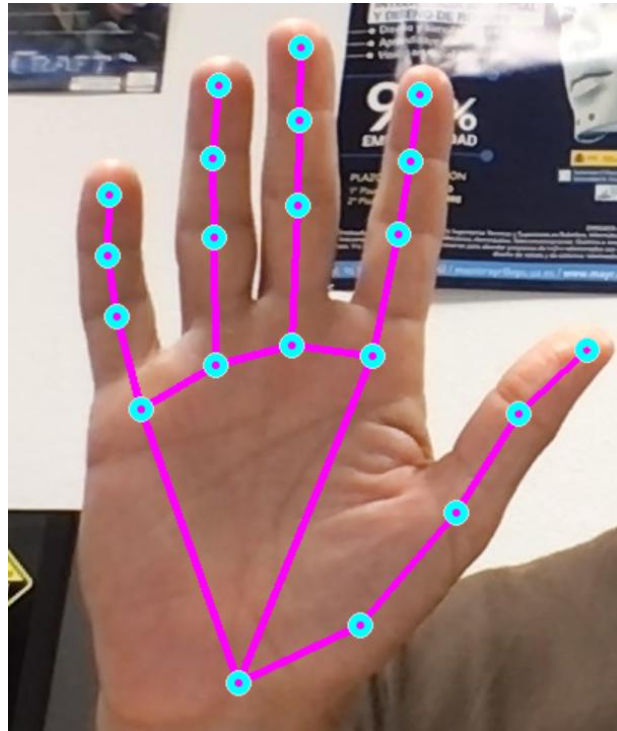
    #image = cv2.imread("C:/path absoluto/ConManos.jpg")
    image = cv2.imread("./Imagenes/ConManos.jpg")
    height, width, _ = image.shape
    image = cv2.flip(image, 1)
    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    results = hands.process(image_rgb)
    print('Handedness:', results.multi_handedness)

    if results.multi_hand_landmarks is not None:
        # Dibujando los puntos y las conexiones mediante mp_drawing
        for hand_landmarks in results.multi_hand_landmarks:
            mp_drawing.draw_landmarks(
                image, hand_landmarks, mp_hands.HAND_CONNECTIONS,
                mp_drawing.DrawingSpec(color=(50,50,255), thickness=4, circle_radius=5),
                mp_drawing.DrawingSpec(color=(255,0,0), thickness=4))

    image = cv2.flip(image, 1)
    cv2.imshow("Imagen", image)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

Utiliza un código similar al del ejemplo anterior con varias imágenes donde aparezcan manos y donde no, observando los resultados y asegurándonos que todo funciona correctamente.



Una vez verificamos que todo funciona correctamente y como el propósito final de la practica es que la interacción humano-ordenador se realice mediante gestos hemos de analizar en tiempo real las imágenes que adquiere la cámara de nuestro portátil. Para esto modificamos el ejemplo anterior para que analice los fotogramas que adquiere la cámara como se muestra en el siguiente ejemplo, observa que hemos cambiado los parámetros que utiliza MediaPipe como se muestra en el recuadro rojo del código. Revisa la documentación de MediaPipe para asegurarte que entiendes como afecta cada uno de estos parámetros.

```

import cv2
import mediapipe as mp

mp_drawing = mp.solutions.drawing_utils
mp_hands = mp.solutions.hands

cap = cv2.VideoCapture(0)

with mp_hands.Hands(
    static_image_mode=False,
    max_num_hands=2,
    min_detection_confidence=0.5) as hands:

    while True:
        ret, frame = cap.read()
        if ret == False:
            break

        height, width, _ = frame.shape
        frame = cv2.flip(frame, 1)
        frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

        results = hands.process(frame_rgb)

        if results.multi_hand_landmarks is not None:
            for hand_landmarks in results.multi_hand_landmarks:
                mp_drawing.draw_landmarks(
                    frame, hand_landmarks, mp_hands.HAND_CONNECTIONS)

        cv2.imshow("Imagen", frame)
        if cv2.waitKey(1) & 0xFF == 27:
            break

cap.release()
cv2.destroyAllWindows()

```

Actividad

Actividad

Utilizando como base un minijuego desarrollado en Python (se recomienda utilizar juegos desarrollados con PyGame por su sencillez) modifícalo para que la interacción con el jugador se realice mediante gestos con las manos. En la valoración de la práctica el juego es secundario, lo que se valorará es la complejidad y originalidad de los gestos empleados para su control. Se entregará tanto el código fuente como una pequeña memoria explicando el juego.



Entregables

- **Memoria:** Se entregará un documento en formato PDF que contenga portada, introducción, metodología y resultados. En caso de utilizar referencias bibliográficas, estas deberán estar correctamente citadas y referenciadas según el formato APA. El nombre del archivo deberá ser el siguiente: *Memoria_EquipoX*.
- **Código:** Se adjuntará al PDF el código utilizado. Al inicio de cada archivo de código deberá incluirse, como comentario, el número del equipo y los nombres de los integrantes. El nombre del archivo deberá ser el siguiente: *Codigo_EquipoX*. En caso de utilizar varios scripts, cada uno deberá llevar un nombre que indique claramente su propósito. Por ejemplo, si se trata del script correspondiente al juego, deberá llamarse *juego*; si corresponde al *control*, deberá llamarse *control*. Todos los scripts y archivos relacionados deberán estar dentro de una carpeta cuyo nombre sea *Codigo_EquipoX*.

Los documentos deberán entregarse comprimidos en un único archivo ZIP a través de UACloud, cuyo nombre deberá ser: *GrupoX_EquipoX*.

- **Evaluación del equipo:** Se deberá completar el documento de evaluación proporcionado por el profesor de forma individual (UACloud) siguiendo las indicaciones correspondientes. Este deberá entregarse en formato PDF con el siguiente nombre: *Ev_Apellido1_Apellido2_Nombre*.